MODULE

06

Remotes & GitHub

Share work with a team: clone, push, pull, fetch, and force-push safely.

YOU WILL LEARN

clone, push, pull	01_clone_push_pull.md
fetch vs pull	02_fetch_vs_pull.md
Rejected pushes & force	03_rejected_push_and_force.md
Exercise: sync with your team	ex06_sync_with_team/
Interview Q&A	INTERVIEW.md

WHY REMOTES

Your laptop is not a team

THE PROBLEM

Commits on your laptop help nobody: the team can’t review them, CI can’t test them, and if the disk dies they’re gone.

THE IDEA

A remote (GitHub, GitLab, Azure DevOps) is the shared copy. You push your commits to it and pull everyone else’s.

PICTURE IT REMOTES

A shared bookshelf

📚 Everyone has a full copy at home. You **push** new pages to the shelf and **pull** pages others added. **origin/main** is your note of what the shelf looked like the last time you checked.

`git clone`
copy the whole shelf home

→

`git push`
put your new pages on the shelf

→

`git fetch`
check the shelf, update origin/*

→

`git pull`
fetch + bring the pages into your branch

TRY IT REMOTES

Connect a project and push it

```
$ git init --bare ../github/shop.git
Initialized empty Git repository in ~/github/shop.git/
$ git remote add origin ../github/shop.git
$ git remote -v
origin  ../github/shop.git (fetch)
origin  ../github/shop.git (push)
$ git push -u origin main
branch 'main' set up to track 'origin/main'.
To ../github/shop.git
 * [new branch]      main -> main
$ git status -sb
## main...origin/main
```

1. A bare repo plays GitHub (offline).
2. **-u** links main to origin/main for later pushes/pulls.
3. Real URL: **git@github.com:you/shop.git**.

TRY IT FETCH VS PULL

fetch first, look, then integrate

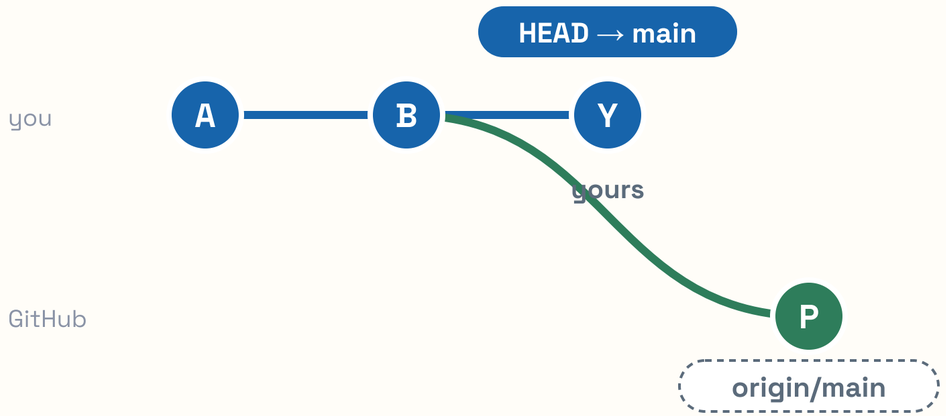
```
$ git fetch
From ../github/shop
   7cb5715..a04f8da  main       -> origin/main
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git log --oneline main..origin/main
a04f8da Add payments
$ git pull --rebase
Rebasing (1/1)
Successfully rebased and updated refs/heads/main.
$ git log --oneline --graph
* b16c859 Add shipping
* a04f8da Add payments
* 7cb5715 Initial commit
$ git push
To ../github/shop.git
   a04f8da..b16c859  main -> main
```

1. **fetch** moves only origin/main.
2. **ahead 1, behind 1**: both sides have new work.
3. **pull --rebase**: linear, no merge commit.

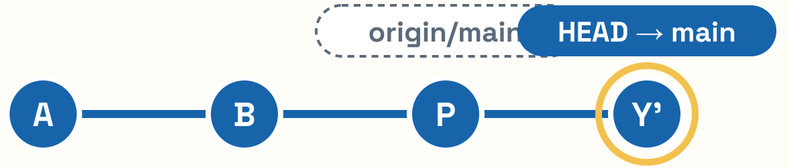
PICTURE IT REMOTES

ahead 1, behind 1

After git fetch



`git pull --rebase`



Your commit is replayed on top of Priya's. Now **git push** works and history stays a straight line.

STEP BY STEP REMOTES

Remote commands cheat sheet

COMMAND	DOES
`git clone <url>`	copy a repository with all history
`git remote -v` / `add` / `remove`	manage remote names and URLs
`git fetch --prune`	download new commits, drop deleted remote branches
`git pull --rebase`	fetch + rebase your commits on top
`git push -u origin feature/x`	publish a new branch and track it
`git push origin --delete feature/x`	delete a remote branch

TRY IT REJECTED PUSH

When the remote has work you don’t

```
$ git push
To ../github/shop.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to '../github/shop.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
$ git pull --rebase
From ../github/shop
   7cb5715..a04f8da  main       -> origin/main
Rebasing (1/1)
Successfully rebased and updated refs/heads/main.
$ git push
To ../github/shop.git
   a04f8da..c989af9  main -> main
```

1. Git refuses to overwrite Priya’s commit.
2. Integrate first, then push.
3. Never ”fix” this with --force on a shared branch.

COMMON MISTAKE FORCE PUSH

--force on a shared branch

✗ WRONG

```
git rebase -i HEAD~3      # on main
git push --force
# Priya's pushed commits: gone from GitHub
```

--force overwrites the remote no matter what's there. Teammates' work disappears from the server.

✓ RIGHT

```
git switch feature/cart     # your own branch
git rebase origin/main
git push --force-with-lease
# refuses if someone pushed meanwhile
```

Only force-push your own branch, and always with **--force-with-lease**. Protect main on GitHub.

IN REAL JOBS REMOTES

Remotes at work

Clone with SSH

```
ssh-keygen -t ed25519 -C "you@company.com"
git clone git@github.com:company/shop.git
```

Publish your branch

```
git push -u origin feature/SHOP-142
```

Contribute via a fork

```
git remote add upstream git@github.com:org/proj.git
git fetch upstream && git rebase upstream/main
```

Keep things tidy

```
git fetch --prune
git branch -vv
```

INTERVIEW SAY THIS IN THE INTERVIEW

“git fetch vs git pull?”

- **fetch** downloads new commits and updates remote-tracking branches like **origin/main**; my branches and files don’t change.
- **pull** = fetch + merge (or rebase with **--rebase**) into my current branch.
- I fetch first and look (**git log main..origin/main**), then integrate.
- A rejected push means the remote has work I don’t have: pull --rebase, then push.

> **Follow-up they ask next:** ”--force vs --force-with-lease?” (with-lease refuses to overwrite commits you haven’t seen; never force shared branches).

RECAP REMEMBER THESE

Module 6 in 5 lines

1. A remote = a shared copy; **origin** = the one you cloned.
2. **push -u** once, then **push** / **pull**.
3. **fetch** updates origin/*; **pull** = fetch + merge/rebase.
4. Rejected push → **pull --rebase** → push.
5. Force only your own branch, with **--force-with-lease**.

PRACTICE NOW

Do exercises/ex06_sync_with_team/: get Priya's commit and push yours.